COMPREHENSIVE_COMPLETION_PLAN

HelixCode Comprehensive Completion Plan

Executive Summary

This document provides a complete audit of unfinished work in the HelixCode project and a detailed phased implementation plan to achieve: - 100% test coverage with NO skipped tests - Complete documentation for all modules - Full user manuals and tutorials - Complete video course production - Updated website content

Generated: January 8, 2026

PART 1: UNFINISHED WORK AUDIT

1.1 Code Implementation Gaps

Critical: Placeholder Implementations (15+ locations)

File	Line	Issue	Priority
`internal/cognee/performance_optimizer.go`	893	`getCPUUsage()` returns `0.0` placeholder	HIGH
`internal/cognee/performance_optimizer.go`	898	`getGPUUsage()` returns `0.0` placeholder	HIGH
`internal/memory/cognee_integration.go`	114	Placeholder for Cognee API (StoreMemory)	HIGH
`internal/memory/cognee_integration.go`	133	Placeholder for Cognee API (RetrieveMemory)	HIGH
`internal/memory/cognee_integration.go`	156	Placeholder for Cognee API (GetContext)	HIGH
`internal/memory/cognee_integration.go`	176	Placeholder for Cognee API (GetSystemInfo)	HIGH

File	Line	Issue	Priority
`internal/memory/cognee_integration.go`	193	Placeholder for Cognee API (GetOptimizationRecommendations)	HIGH
`internal/memory/cognee_integration.go`	216	Placeholder for optimization application	HIGH
`internal/memory/cognee_integration.go`	229	Placeholder for health check	HIGH
`internal/workflow/planmode/executor.go`	367	`executeFileOperation()` placeholder	MEDIUM
`internal/workflow/planmode/executor.go`	388	`executeCodeGeneration()` placeholder	MEDIUM
`internal/workflow/planmode/executor.go`	394	`executeCodeAnalysis()` placeholder	MEDIUM
`internal/workflow/planmode/executor.go`	400	`executeValidation()` placeholder	MEDIUM
`internal/workflow/planmode/executor.go`	406	`executeTesting()` placeholder	MEDIUM
`internal/llm/usage_analytics.go`	566-583	Placeholder analytics data	MEDIUM
`internal/config/config.go`	1464, 1482	Placeholder save implementations	LOW

Mock Implementations Needing Real Integration

Package	Component	Status	Action Required
`memory/providers/faiss_provider.go`	Vector search	Mock	Integrate real FAISS library
`memory/providers/mem0_provider.go`	Memory operations	Stub	Implement Mem0 API calls
`memory/providers/character_ai_provider.go`	AI interface	Mock	Implement CharacterAI API
`memory/providers/weaviate_provider.go`	Backup/Restore	Not implemented	Use Weaviate backup API
`memory/providers/zep_provider.go`	Embedding search	Placeholder	Implement real search

TODO/FIXME Items in Code

Location	Content	Category
`internal/workflow/executor.go:633`	Go template implementation	Code Generation
`internal/workflow/executor.go:644`	Node.js template implementation	Code Generation
`internal/workflow/executor.go:673`	Rust template implementation	Code Generation
`internal/server/handlers.go:286`	User authentication placeholder	Security

Missing API Endpoints (E2E Tests Expecting)

Endpoint	Test File	Status
`/api/v1/llm/providers`	`tests/e2e/phase2/real_server_test.go:213`	Not implemented
`/api/v1/server/info`	`tests/e2e/phase2/real_server_test.go:281`	Not implemented
`/api/v1/metrics`	`tests/e2e/phase2/real_server_test.go:298`	Not implemented
`/api/v1/memory/systems`	`tests/e2e/phase3/production_validation_test.go:141`	Not implemented

1.2 Skipped Tests Analysis (MUST BE FIXED)

Database-Related Skips (19 tests)

Location: `internal/task/manager_test.go`

Tests skipped due to "Checkpoint tests require real database": 1. `TestCheckpointCreation` 2. `TestCheckpointRestore` 3. `TestCheckpointWithDependencies` 4. `TestCheckpointPersistence` 5. `TestCheckpointRecovery` 6. `TestTaskManagerWithDB` 7. `TestTaskPersistence` 8. `TestTaskRetrieval` 9. `TestTaskUpdate` 10. `TestTaskDeletion` 11. `TestTaskDependencies` 12. `TestTaskCheckpointing` 13. `TestBulkTaskOperations` 14. `TestConcurrentTaskAccess` 15. `TestSplitTask` (requires SplitStrategy) 16-19. Additional database integration tests

Solution: Provide PostgreSQL container in test infrastructure.

Browser Tests (12+ tests)

Location: `internal/tools/browser/browser_test.go`

Tests skipped due to "Chrome not installed": - `TestBrowserNavigation` - `TestBrowserScreenshot` - `TestBrowserClick` - `TestBrowserType` - `TestBrowserWait` - `TestBrowserExtract` - `TestBrowserPDF` - `TestBrowserCookies` - `TestBrowserLocalStorage` - `TestBrowserExecuteScript` - `TestBrowserNetwork` - `TestBrowserMultiTab`

Solution: Provide Chrome/Chromium in test container.

SSH/Worker Tests (3+ tests)

Location: `internal/worker/distributed_manager_test.go`, `ssh_security_test.go`

Tests skipped due to "Requires SSH setup" or "Requires root privileges": - `TestDistributedWorkerManager` - `TestSSHWorkerPool` - `TestSSHSecurity`

Solution: Provide SSH server container with test keys.

Network Tests (3 tests)

Location: `internal/discovery/broadcast_test.go`

Tests skipped due to "UDP multicast unreliable in test environment": - `TestBroadcastDiscovery` - `TestMulticastListener` - `TestNetworkAnnouncement`

Solution: Use Docker network with multicast support.

Voice/Audio Tests

Location: `internal/tools/voice/voice_test.go`

Tests skipped due to "no devices available": - `TestDeviceManager_ListDevices` - `TestVoiceRecording`

Solution: Use virtual audio device in container (PulseAudio null sink).

LLM Provider Integration Tests (35+ tests)

Location: `internal/llm/integration_test.go`, `local_providers_integration_test.go`

Tests conditionally skipped when providers unavailable: - Ollama tests (5+) - Llama.cpp tests (3+) - vLLM tests (3+) - LocalAI tests (3+) - FastChat tests (3+) - TextGen tests (3+) - LM Studio tests (3+) - Jan AI tests (2+) - KoboldAI tests (2+) - GPT4All tests (2+) - TabbyAPI tests (2+) - MLX tests (2+) - MistralRS tests (2+)

Solution: Provide Ollama container with test model for core tests.

Cloud Provider Tests (8+ tests)

Tests skipped due to missing API keys: - AWS Bedrock tests - OpenAI tests - Anthropic tests - Gemini tests - Azure OpenAI tests

Solution: Provide mock server container OR require API keys in CI secrets.

1.3 Application Completion Status

Terminal UI (`applications/terminal_ui/`)

- **Status**: 70% complete
- **Missing**:
  - Worker Management implementation
  - Project Management implementation
  - Session Management implementation
  - LLM Interaction implementation
  - Full Cognee integration
- **Tests**: Basic (134 lines) - needs expansion

Desktop (`applications/desktop/`)

- **Status**: 80% complete
- **Missing**:
  - Projects tab (placeholder)
  - Sessions tab (placeholder)
  - LLM tab (placeholder)
  - Real worker data integration
- **Tests**: Theme-focused only (125 lines)

Aurora OS (`applications/aurora_os/`)

- **Status**: 75% complete
- **Missing**:
  - Projects/Sessions/LLM tabs
  - Real diagnostics implementation
  - Actual system metrics (currently simulated)

- **Tests**: Minimal (39 lines) - NEEDS SIGNIFICANT EXPANSION

Harmony OS (`applications/harmony_os/`)

- **Status**: 85% complete (BEST)
- **Missing**:
  - Worker registration
  - Task creation dialogs
  - Some Aurora features
- **Tests**: Most comprehensive (254 lines)

iOS (`applications/ios/`)

- **Status**: 30% complete
- **Missing**:
  - Navigation controller
  - Task detail views
  - Project management
  - Worker management
  - Settings
  - Error handling
- **Tests**: NONE - NEEDS FULL TEST SUITE

Android (`applications/android/`)

- **Status**: 20% complete
- **Missing**:
  - Activity layouts
  - Navigation
  - All feature implementations
  - TaskAdapter implementation
- **Tests**: NONE - NEEDS FULL TEST SUITE

1.4 Documentation Gaps

Package Documentation: 100% COMPLETE

All 40 internal packages have README files.

Minor Documentation Improvements Needed:

- `internal/focus/README.md` - Add more examples
- `internal/logo/README.md` - Minimal, could expand

- Cross-package references could be improved

1.5 Video Course Status

Course Platform: COMPLETE

- Player infrastructure: 100%
- Progress tracking: 100%
- Certificate system: 100%

Video Content: NOT PRODUCED

- **Scripts Written**: 12+ Phase 3 scripts COMPLETE
- **Actual Videos**: 0% - Using placeholder videos (BigBuckBunny, etc.)
- **Videos Needed**: 28-32 videos across 4 courses
- **Total Duration Needed**: ~10+ hours of content

1.6 Website Status

Main Website (`github_pages_website/`): 95% COMPLETE

- Version: v1.1.0 (November 2025)
- All sections implemented
- Mobile responsive
- Dark/Light theme

Missing/Planned:

- Interactive demo playground
- Multi-language support (i18n)
- Real video content in courses section
- PDF certificate download (jsPDF needed)

Secondary Website (`website/`): INCOMPLETE

- Only planning document exists
- Hugo/Docusaurus not implemented
- May be deprecated in favor of Github-Pages-Website

PART 2: TEST INFRASTRUCTURE FOR ZERO SKIPPED TESTS

2.1 Complete Docker Compose Test Environment

```yaml
# docker-compose.full-test.yml
# Complete test infrastructure ensuring NO tests are skipped

version: '3.8'

services:
  # ============================================
  # DATABASE SERVICES
  # ============================================
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: helixcode
      POSTGRES_PASSWORD: helixcode_test_password
      POSTGRES_DB: helixcode_test
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U helixcode -d
        helixcode_test"]
      interval: 5s
      timeout: 5s
      retries: 5
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - helixcode-test

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 5
```

```yaml
  networks:
    - helixcode-test

# ============================================
# LLM PROVIDERS
# ============================================
ollama:
  image: ollama/ollama:latest
  ports:
    - "11434:11434"
  volumes:
    - ollama_data:/root/.ollama
  environment:
    - OLLAMA_HOST=0.0.0.0
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:11434/api/
      tags"]
    interval: 10s
    timeout: 10s
    retries: 10
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: all
            capabilities: [gpu]
  networks:
    - helixcode-test

# Mock LLM server for cloud provider tests
mock-llm-server:
  build:
    context: ./tests/e2e/mocks
    dockerfile: Dockerfile.llm-mock
  ports:
    - "8090:8090"
  environment:
    - MOCK_OPENAI=true
    - MOCK_ANTHROPIC=true
    - MOCK_GEMINI=true
    - MOCK_AZURE=true
    - MOCK_BEDROCK=true
  healthcheck:
```

```yaml
    test: ["CMD", "curl", "-f", "http://localhost:8090/health"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - helixcode-test

# ============================================
# BROWSER TESTING
# ============================================
chrome:
  image: selenium/standalone-chrome:latest
  ports:
    - "4444:4444"
    - "7900:7900"
  shm_size: 2gb
  environment:
    - SE_NODE_MAX_SESSIONS=5
    - SE_NODE_SESSION_TIMEOUT=300
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:4444/status"]
    interval: 10s
    timeout: 10s
    retries: 5
  networks:
    - helixcode-test

# Chromedp-compatible Chrome
chromedp:
  image: chromedp/headless-shell:latest
  ports:
    - "9222:9222"
  networks:
    - helixcode-test

# ============================================
# SSH WORKER TESTING
# ============================================
ssh-server:
  build:
    context: ./tests/infrastructure
    dockerfile: Dockerfile.ssh-server
  ports:
    - "2222:22"
```

```yaml
    environment:
      - SSH_USER=helixcode
      - SSH_PASSWORD=test_password
    volumes:
      - ./tests/infrastructure/ssh_keys:/home/helixcode/.ssh:ro
    healthcheck:
      test: ["CMD", "nc", "-z", "localhost", "22"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - helixcode-test

  ssh-worker-1:
    build:
      context: ./tests/infrastructure
      dockerfile: Dockerfile.ssh-worker
    ports:
      - "2223:22"
    environment:
      - WORKER_ID=worker-1
      - SSH_USER=helixcode
    networks:
      - helixcode-test

  ssh-worker-2:
    build:
      context: ./tests/infrastructure
      dockerfile: Dockerfile.ssh-worker
    ports:
      - "2224:22"
    environment:
      - WORKER_ID=worker-2
      - SSH_USER=helixcode
    networks:
      - helixcode-test

  # ============================================
  # AUDIO/VOICE TESTING
  # ============================================
  pulseaudio:
    image: thewierdnut/pulseaudio:latest
    environment:
      - PULSE_SERVER=unix:/run/pulse/native
```

```yaml
    volumes:
      - pulse_socket:/run/pulse
    networks:
      - helixcode-test

  # ============================================
  # NETWORK TESTING (Multicast Support)
  # ============================================
  multicast-router:
    image: alpine:latest
    cap_add:
      - NET_ADMIN
    command: >
      sh -c "
        apk add --no-cache iproute2 &&
        sysctl -w net.ipv4.ip_forward=1 &&
        ip route add 224.0.0.0/4 dev eth0 &&
        tail -f /dev/null
      "
    networks:
      helixcode-test:
        ipv4_address: 172.28.0.254

  # ============================================
  # KNOWLEDGE GRAPH / MEMORY
  # ============================================
  cognee:
    image: topoteretes/cognee:latest
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://
        helixcode:helixcode_test_password@postgres:5432/cognee
    depends_on:
      postgres:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 10s
      timeout: 10s
      retries: 10
    networks:
      - helixcode-test
```

```yaml
  weaviate:
    image: semitechnologies/weaviate:1.24.1
    ports:
      - "8081:8080"
    environment:
      - QUERY_DEFAULTS_LIMIT=25
      - AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED=true
      - PERSISTENCE_DATA_PATH=/var/lib/weaviate
      - DEFAULT_VECTORIZER_MODULE=none
    volumes:
      - weaviate_data:/var/lib/weaviate
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/
        v1/.well-known/ready"]
      interval: 10s
      timeout: 10s
      retries: 5
    networks:
      - helixcode-test

  # ============================================
  # HELIXCODE SERVER (Test Instance)
  # ============================================
  helixcode-server:
    build:
      context: .
      dockerfile: Dockerfile.test
    ports:
      - "8080:8080"
    environment:
      - HELIX_AUTH_JWT_SECRET=test-jwt-secret-for-testing-only
      - HELIX_DATABASE_HOST=postgres
      - HELIX_DATABASE_PORT=5432
      - HELIX_DATABASE_USER=helixcode
      - HELIX_DATABASE_PASSWORD=helixcode_test_password
      - HELIX_DATABASE_NAME=helixcode_test
      - HELIX_REDIS_HOST=redis
      - HELIX_REDIS_PORT=6379
      - OLLAMA_HOST=http://ollama:11434
      - CHROME_DEBUGGER_URL=ws://chromedp:9222
      - SSH_TEST_HOST=ssh-server
      - SSH_TEST_PORT=22
      - COGNEE_ENDPOINT=http://cognee:8000
      - WEAVIATE_ENDPOINT=http://weaviate:8080
```

```
      - MOCK_LLM_ENDPOINT=http://mock-llm-server:8090
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
      ollama:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 10s
      timeout: 10s
      retries: 10
    networks:
      - helixcode-test

networks:
  helixcode-test:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16

volumes:
  postgres_data:
  ollama_data:
  weaviate_data:
  pulse_socket:
```

2.2 Supporting Dockerfiles

Dockerfile.ssh-server

```
# tests/infrastructure/Dockerfile.ssh-server
FROM alpine:3.19

RUN apk add --no-cache openssh-server openssh-client bash curl

RUN mkdir -p /var/run/sshd && \
    ssh-keygen -A && \
    adduser -D -s /bin/bash helixcode && \
    echo "helixcode:test_password" | chpasswd && \
    mkdir -p /home/helixcode/.ssh && \
    chown -R helixcode:helixcode /home/helixcode/.ssh
```

```
COPY ssh_keys/id_rsa.pub /home/helixcode/.ssh/authorized_keys
RUN chown helixcode:helixcode /home/helixcode/.ssh/
        authorized_keys && \
    chmod 600 /home/helixcode/.ssh/authorized_keys

RUN echo "PermitRootLogin no" >> /etc/ssh/sshd_config && \
    echo "PasswordAuthentication yes" >> /etc/ssh/sshd_config &&
        \
    echo "PubkeyAuthentication yes" >> /etc/ssh/sshd_config

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D", "-e"]
```

Dockerfile.ssh-worker

```
# tests/infrastructure/Dockerfile.ssh-worker
FROM golang:1.24-alpine

RUN apk add --no-cache openssh-server bash curl git

RUN mkdir -p /var/run/sshd && \
    ssh-keygen -A && \
    adduser -D -s /bin/bash helixcode && \
    echo "helixcode:test_password" | chpasswd && \
    mkdir -p /home/helixcode/.ssh /home/helixcode/workspace

COPY ssh_keys/id_rsa.pub /home/helixcode/.ssh/authorized_keys
RUN chown -R helixcode:helixcode /home/helixcode

# Pre-install Go tools for worker
RUN go install golang.org/x/tools/gopls@latest && \
    go install github.com/golangci/golangci-lint/cmd/golangci-
        lint@latest

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D", "-e"]
```

Dockerfile.llm-mock

```
# tests/e2e/mocks/Dockerfile.llm-mock
FROM golang:1.24-alpine AS builder
```

```
WORKDIR /app
COPY . .
RUN go build -o mock-llm-server ./cmd/mock-server

FROM alpine:3.19
COPY --from=builder /app/mock-llm-server /usr/local/bin/
EXPOSE 8090
CMD ["mock-llm-server"]
```

2.3 Test Environment Setup Script

```
#!/bin/bash
# scripts/setup-full-test-env.sh

set -e

echo "=== HelixCode Full Test Environment Setup ==="

# 1. Generate SSH keys if not exists
if [ ! -f tests/infrastructure/ssh_keys/id_rsa ]; then
    echo "Generating SSH test keys..."
    mkdir -p tests/infrastructure/ssh_keys
    ssh-keygen -t rsa -b 4096 -f tests/infrastructure/ssh_keys/
        id_rsa -N "" -C "helixcode-test"
fi

# 2. Pull and start all containers
echo "Starting full test infrastructure..."
docker compose -f docker-compose.full-test.yml up -d

# 3. Wait for all services to be healthy
echo "Waiting for services to be healthy..."
services=("postgres" "redis" "ollama" "chrome" "ssh-server"
        "cognee" "weaviate" "mock-llm-server")
for service in "${services[@]}"; do
    echo "Waiting for $service..."
    timeout 120 bash -c "until docker compose -f docker-
        compose.full-test.yml ps $service | grep -q healthy; do
        sleep 2; done"
done

# 4. Pull Ollama model for tests
echo "Pulling Ollama test model (llama2:7b)..."
```

```
docker compose
        -f docker-compose.full-test.yml exec ollama ollama pull
        llama2:7b

# 5. Initialize database schema
echo "Initializing database schema..."
docker compose -f docker-compose.full-test.yml exec helixcode-
        server /app/bin/helixcode migrate

# 6. Verify all services
echo "Verifying services..."
curl -sf http://localhost:5432 || echo "PostgreSQL: OK (port
        check via psql)"
curl -sf http://localhost:6379 || echo "Redis: OK (check via
        redis-cli)"
curl -sf http://localhost:11434/api/tags && echo "Ollama: OK"
curl -sf http://localhost:4444/status && echo "Selenium Chrome:
        OK"
curl -sf http://localhost:8080/health && echo "HelixCode Server:
        OK"
curl -sf http://localhost:8090/health && echo "Mock LLM Server:
        OK"

echo ""
echo "=== Full Test Environment Ready ==="
echo "Run tests with: make test-full"
```

2.4 Environment Variables for Full Testing

```
# .env.full-test
# Complete environment for zero-skip testing

# Database
HELIX_DATABASE_HOST=localhost
HELIX_DATABASE_PORT=5432
HELIX_DATABASE_USER=helixcode
HELIX_DATABASE_PASSWORD=helixcode_test_password
HELIX_DATABASE_NAME=helixcode_test
HELIX_DATABASE_ENABLED=true

# Redis
HELIX_REDIS_HOST=localhost
HELIX_REDIS_PORT=6379
HELIX_REDIS_ENABLED=true
```

```
# Authentication
HELIX_AUTH_JWT_SECRET=test-jwt-secret-for-testing-only-32chars

# LLM Providers
OLLAMA_HOST=http://localhost:11434
MOCK_LLM_ENDPOINT=http://localhost:8090
OPENAI_API_KEY=mock-openai-key
ANTHROPIC_API_KEY=mock-anthropic-key
GEMINI_API_KEY=mock-gemini-key

# Browser Testing
CHROME_DEBUGGER_URL=ws://localhost:9222
SELENIUM_URL=http://localhost:4444

# SSH Workers
SSH_TEST_HOST=localhost
SSH_TEST_PORT=2222
SSH_TEST_USER=helixcode
SSH_TEST_KEY_PATH=./tests/infrastructure/ssh_keys/id_rsa

# Memory Providers
COGNEE_ENDPOINT=http://localhost:8000
WEAVIATE_ENDPOINT=http://localhost:8081

# Test Configuration
SKIP_EXPENSIVE_TESTS=false
SKIP_HARDWARE_TESTS=false
TESTING_SHORT=false
CI_MODE=true
TEST_TIMEOUT=30m
PARALLEL_JOBS=4
```

PART 3: PHASED IMPLEMENTATION PLAN

Phase 1: Test Infrastructure (Week 1-2)

1.1 Container Infrastructure Setup

- [ ] Create `docker-compose.full-test.yml` with all services

- [ ] Create `Dockerfile.ssh-server` for SSH testing
- [ ] Create `Dockerfile.ssh-worker` for distributed worker tests
- [ ] Create `Dockerfile.llm-mock` for cloud provider mocking
- [ ] Create SSH key generation scripts
- [ ] Create environment setup scripts
- [ ] Verify all containers work together

1.2 Mock Server Implementation

- [ ] Implement OpenAI-compatible mock server
- [ ] Implement Anthropic mock endpoints
- [ ] Implement Gemini mock endpoints
- [ ] Implement Azure OpenAI mock endpoints
- [ ] Implement AWS Bedrock mock endpoints
- [ ] Add configurable response delays and errors

1.3 Test Configuration Updates

- [ ] Update `config/test-config.yaml` for container endpoints
- [ ] Create `.env.full-test` with all required variables
- [ ] Update CI/CD pipeline to use full test infrastructure
- [ ] Add health check verification in test setup

Phase 2: Fix All Skipped Tests (Week 2-4)

2.1 Database Tests

- [ ] Update `internal/task/manager_test.go` to use containerized PostgreSQL
- [ ] Remove all “requires real database” skip conditions
- [ ] Add database connection retry logic in tests
- [ ] Verify all 19 checkpoint tests pass

2.2 Browser Tests

- [ ] Update `internal/tools/browser/browser_test.go` to use Selenium/chromedp container
- [ ] Configure Chrome remote debugging URL from environment
- [ ] Remove “Chrome not installed” skip conditions
- [ ] Verify all 12+ browser tests pass

2.3 SSH/Worker Tests

- [ ] Update `internal/worker/distributed_manager_test.go` to use SSH container
- [ ] Update `internal/worker/ssh_security_test.go` to use SSH container
- [ ] Configure SSH connection from environment variables
- [ ] Remove “requires SSH setup” skip conditions
- [ ] Verify all worker tests pass

2.4 Network Tests

- [ ] Configure Docker network with multicast support

- [ ] Update `internal/discovery/broadcast_test.go` to use test network
- [ ] Remove “flaky network test” skip conditions
- [ ] Verify all discovery tests pass

2.5 Voice/Audio Tests

- [ ] Configure PulseAudio virtual device in container
- [ ] Update `internal/tools/voice/voice_test.go` to use virtual device
- [ ] Remove “no devices available” skip conditions
- [ ] Verify voice tests pass (or document hardware requirement)

2.6 LLM Provider Tests

- [ ] Configure Ollama container with test model
- [ ] Update integration tests to use mock server for cloud providers
- [ ] Remove all “provider not available” skip conditions
- [ ] Verify all 35+ LLM tests pass

Phase 3: Implement Missing Code (Week 4-8)

3.1 Cognee Integration (HIGH PRIORITY)

- [ ] Implement `getCPUUsage()` in `performance_optimizer.go`
- [ ] Implement `getGPUUsage()` in `performance_optimizer.go`
- [ ] Implement `StoreMemory()` API call in `cognee_integration.go`
- [ ] Implement `RetrieveMemory()` API call
- [ ]

- [ ] Implement `GetContext()` API call
- [ ] Implement `GetSystemInfo()` API call
- [ ] Implement `GetOptimizationRecommendations()` API call
- [ ] Implement health check with real Cognee
- [ ] Add comprehensive tests for all Cognee functions

3.2 Workflow Plan Mode (MEDIUM PRIORITY)

- [ ] Implement `executeFileOperation()` with real file operations
- [ ] Implement `executeCodeGeneration()` with LLM integration
- [ ] Implement `executeCodeAnalysis()` with code analysis tools
- [ ] Implement `executeValidation()` with validation checks
- [ ] Implement `executeTesting()` with test runner
- [ ] Add tests for all plan mode operations

3.3 Memory Providers

- [ ] Complete FAISS provider implementation (or document as optional)
- [ ] Complete Mem0 provider implementation
- [ ] Complete CharacterAI provider implementation
- [ ] Implement Weaviate backup/restore using API
- [ ] Implement Zep embedding search
- [ ] Add tests for all memory providers

3.4 Usage Analytics

- [ ] Implement real provider usage tracking

- [ ] Implement success rate calculation
- [ ] Implement trend analysis
- [ ] Add tests for analytics

3.5 Missing API Endpoints

- [ ] Implement `/api/v1/llm/providers` endpoint
- [ ] Implement `/api/v1/server/info` endpoint
- [ ] Implement `/api/v1/metrics` endpoint
- [ ] Implement `/api/v1/memory/systems` endpoint
- [ ] Update E2E tests to use new endpoints

3.6 Code Generation Templates

- [ ] Implement Go template in workflow executor
- [ ] Implement Node.js template
- [ ] Implement Rust template
- [ ] Add tests for all templates

Phase 4: Application Completion (Week 8-12)

4.1 Terminal UI

- [ ] Implement Worker Management view
- [ ] Implement Project Management view
- [ ] Implement Session Management view
- [ ]

- [ ] Implement LLM Interaction view
- [ ] Complete Cognee integration
- [ ] Add comprehensive tests (target: 500+ lines)

4.2 Desktop Application

- [ ] Implement Projects tab functionality
- [ ] Implement Sessions tab functionality
- [ ] Implement LLM tab functionality
- [ ] Connect to real worker data
- [ ] Add comprehensive tests

4.3 Aurora OS Application

- [ ] Implement Projects/Sessions/LLM tabs
- [ ] Implement real diagnostics
- [ ] Connect to real system metrics
- [ ] Expand test coverage (target: 200+ lines)

4.4 Harmony OS Application

- [ ] Implement worker registration
- [ ] Implement task creation dialogs
- [ ] Port relevant Aurora features
- [ ] Verify test coverage

4.5 iOS Application

- [ ]

- [ ] Implement navigation controller
- [ ] Implement task detail views
- [ ] Implement project management
- [ ] Implement worker management
- [ ] Implement settings
- [ ] Add proper error handling
- [ ] Create Swift test suite (XCTest)
- [ ] Target: 80%+ test coverage

4.6 Android Application

- [ ]
- [ ] Create activity layouts (XML)
- [ ] Implement navigation
- [ ] Implement all feature views
- [ ] Complete TaskAdapter implementation
- [ ] Add proper dependency injection
- [ ] Create Kotlin test suite (JUnit/Espresso)
- [ ] Target: 80%+ test coverage

Phase 5: Test Coverage Completion (Week 12-14)

5.1 Test Type Coverage

Unit Tests

- [ ]
- [ ] Ensure every package has `*_test.go` files
- [ ]

- [ ] Minimum 80% line coverage per package
- [ ] All public functions must have tests
- [ ] Mock all external dependencies

Integration Tests

- [ ] Test all database operations
- [ ] Test all Redis operations
- [ ] Test all API endpoints
- [ ] Test all LLM provider integrations
- [ ] Test all memory provider integrations

E2E Tests

- [ ] Complete challenge framework tests
- [ ] All 6 challenge definitions must pass
- [ ] Multi-interface tests (CLI, TUI, REST, WebSocket, Desktop, Mobile)
- [ ] Multi-provider tests

Benchmark Tests

- [ ] Add benchmarks for all critical paths
- [ ] Performance baselines established
- [ ] Regression detection configured

Load Tests

- [ ] Notification load tests
- [ ]

- [ ] API load tests
- [ ] Database load tests
- [ ] Worker pool load tests

Security Tests

- [ ] OWASP test coverage
- [ ] Authentication tests
- [ ] Authorization tests
- [ ] Input validation tests
- [ ] SQL injection prevention tests
- [ ] XSS prevention tests

5.2 Coverage Targets

Package	Current	Target
internal/auth	85%	95%
internal/llm	80%	95%
internal/task	70%	95%
internal/worker	75%	95%
internal/workflow	70%	95%
internal/memory	65%	95%
internal/cognee	60%	95%
internal/tools	75%	95%
internal/editor	90%	95%
internal/server	80%	95%
internal/database	75%	95%
applications/*	30%	80%

Phase 6: Documentation Completion (Week 14-16)

6.1 Package Documentation

- [ ] Expand `internal/focus/README.md` with more examples
- [ ] Expand `internal/logo/README.md`
- [ ] Add cross-package references
- [ ] Document all sub-packages prominently

6.2 API Documentation

- [ ] Complete OpenAPI/Swagger specification
- [ ] Generate API reference from code
- [ ] Add request/response examples
- [ ] Document all error codes

6.3 Architecture Documentation

- [ ] Update architecture diagrams
- [ ] Document data flow
- [ ] Document component interactions
- [ ] Document deployment architectures

6.4 Development Documentation

- [ ] Contributing guide updates
- [ ] Code style guide
- [ ] Testing guide

- [ ] Debugging guide

Phase 7: User Manual Updates (Week 16-17)

7.1 Tutorial Updates

- [ ] Review and update Tutorial 1 (Web App)
- [ ] Review and update Tutorial 2 (Refactoring)
- [ ] Review and update Tutorial 3 (AI Providers)
- [ ] Review and update Tutorial 4 (Browser Automation)
- [ ] Review and update Tutorial 5 (Voice to Code)
- [ ] Review and update Tutorial 6 (Multi-File Edits)
- [ ] Review and update Tutorial 7 (Distributed Development)
- [ ] Review and update Tutorial 8 (Plan Mode)

7.2 New Tutorials

- [ ] Tutorial 9: Mobile Development (iOS/Android)
- [ ] Tutorial 10: Custom Tool Development
- [ ] Tutorial 11: MCP Integration
- [ ] Tutorial 12: Enterprise Deployment

7.3 Reference Updates

- [ ] Update CLI reference
- [ ] Update configuration reference
- [ ]

- [ ] Update API reference
- [ ] Update troubleshooting guide

Phase 8: Video Course Production (Week 17-24)

8.1 Pre-Production (Week 17-18)

- [ ] Set up recording environment (OBS/Camtasia)
- [ ] Configure microphone and audio
- [ ] Prepare demo projects
- [ ] Create slide templates
- [ ] Script review and finalization

8.2 Phase 3 Course Production (Week 18-20)

- [ ] Record Video 1: Phase 3 Overview (10 min)
- [ ] Record Video 2: Getting Started (10 min)
- [ ] Record Video 3: Session Fundamentals (10 min)
- [ ] Record Video 4: Advanced Sessions (10 min)
- [ ] Record Video 5: Memory Basics (10 min)
- [ ] Record Video 6: Advanced Memory (10 min)
- [ ] Record Video 7: Persistence Basics (10 min)
- [ ] Record Video 8: Advanced Persistence (10 min)
- [ ] Record Video 9: Template Basics (10 min)
- [ ] Record Video 10: Advanced Templates (10 min)

- [ ] Record Video 11: Integration Patterns (10 min)
- [ ] Record Video 12: Advanced Workflows (10 min)

8.3 Introduction Course (Week 20-21)

- [ ] Record Welcome to HelixCode (15 min)
- [ ] Record Installation and Setup (20 min)
- [ ] Record Your First Project (25 min)
- [ ] Record Understanding AI Providers (30 min)
- [ ] Record Distributed Computing (35 min)
- [ ] Record Automated Workflows (40 min)

8.4 Advanced Course (Week 21-22)

- [ ] Record MCP Protocol (45 min)
- [ ] Record Mobile Clients (50 min)
- [ ] Record Custom Tools (55 min)
- [ ] Record Performance Optimization (45 min)

8.5 Production Deployment Course (Week 22-23)

- [ ] Record Production Architecture (40 min)
- [ ] Record Docker and Kubernetes (50 min)
- [ ] Record Security Best Practices (45 min)
- [ ] Record Monitoring and Observability (30 min)

8.6 Post-Production (Week 23-24)

- [ ]

- [ ] Edit all videos
- [ ] Add captions/subtitles
- [ ] Create thumbnails
- [ ] Upload to hosting
- [ ] Replace placeholder URLs in course-data.js
- [ ] Generate transcripts
- [ ] Test playback on all platforms

Phase 9: Website Updates (Week 24-26)

9.1 Content Updates

- [ ] Update feature list with new capabilities
- [ ] Update provider list
- [ ] Update platform support matrix
- [ ] Update download links
- [ ] Update documentation links

9.2 Course Integration

- [ ] Replace placeholder video URLs
- [ ] Update course metadata
- [ ] Enable certificate downloads (implement jsPDF)
- [ ] Test all course functionality

9.3 New Features

- [ ]

- [ ] Add interactive demo playground
- [ ] Add live chat widget (optional)
- [ ] Implement multi-language support (i18n)
- [ ] Add PWA support

9.4 Quality Assurance

- [ ]
- [ ] Full link validation
- [ ] Cross-browser testing
- [ ] Mobile responsiveness testing
- [ ] Performance optimization
- [ ] SEO optimization
- [ ] Accessibility audit (WCAG 2.1 AA)

PART 4: TEST TYPES AND FRAMEWORK

4.1 Supported Test Types (6 Types)

Type 1: Unit Tests

- **Framework**: Go testing + testify
- **Location**: `*_test.go` files alongside source
- **Command**: `go test -v ./...`
- **Coverage**: `go test -cover ./...`

Type 2: Integration Tests

- **Framework**: Go testing with build tags
- **Build Tag**: `// +build integration`
- **Location**: `*_integration_test.go` files
- **Command**: `go test -v -tags=integration ./...`

Type 3: E2E Tests

- **Framework**: Custom challenge framework
- **Location**: `tests/e2e/`
- **Components**:
  - Challenge definitions (`tests/e2e/challenges/definitions/`)
  - Core tests (`tests/e2e/core/`)
  - Mock services (`tests/e2e/mocks/`)
  - Orchestrator (`tests/e2e/orchestrator/`)
- **Command**: `go run tests/e2e/challenges/cmd/runner/main.go`

Type 4: Benchmark Tests

- **Framework**: Go testing benchmarks
- **Location**: `*_bench_test.go` or `*_test.go` with `Benchmark*` functions
- **Command**: `go test -bench=. -benchmem ./...`
- **Count**: 129 benchmark functions

Type 5: Load Tests

- **Framework**: Custom load testing
- **Location**: `test/load/`
- **Components**: Notification load tests, API load tests

Type 6: Security Tests

- **Framework**: Custom OWASP-style tests
- **Location**: `tests/security/`
- **Coverage**: OWASP Top 10 vulnerabilities

4.2 Tests Bank Framework

Structure

```
tests/
├── e2e/
│   ├── challenges/
│   │   ├── cmd/runner/main.go      # Test runner
│   │   ├── definitions/            # Challenge JSON files
│   │   ├── executor.go             # Challenge execution
│   │   ├── validator.go            # Code validation
│   │   ├── functional_validator.go # Runtime validation
│   │   ├── runtime_validator.go    # Execution validation
│   │   ├── usecase_validator.go    # Use case validation
```

```
│   │   ├── manager.go              # Challenge management
│   │   ├── types.go                # Type definitions
│   │   └── test-results/           # Generated projects
│   ├── core/                       # Core E2E tests
│   ├── mocks/                      # Mock services
│   └── orchestrator/               # Test orchestration
├── integration/                    # Integration tests
├── security/                       # Security tests
├── automation/                     # Automation tests
└── infrastructure/                 # Test infrastructure
```

Challenge Definitions

- notes-project.json
- url-shortener.json
- cli-task-manager.json
- ascii-art-generator.json
- json-validator-cli.json
- tic-tac-toe-tui.json

Running Tests Bank

```
# List all challenges
go run tests/e2e/challenges/cmd/runner/main.go -list

# Run single challenge
go run tests/e2e/challenges/cmd/runner/main.go -challenge notes-
        project-001

# Run with specific interface
go run tests/e2e/challenges/cmd/runner/main.go -interfaces
        cli,tui

# Run with specific provider
go run tests/e2e/challenges/cmd/runner/main.go -provider ollama

# Run all challenges
go run tests/e2e/challenges/cmd/runner/main.go -all
```

PART 5: MAKEFILE TARGETS

```
# Add to helix_code/Makefile
```

```
# ============================================
# FULL TEST INFRASTRUCTURE
# ============================================

.PHONY: test-infra-up test-infra-down test-infra-status

test-infra-up:
    @echo "Starting full test infrastructure..."
    docker compose -f docker-compose.full-test.yml up -d
    @echo "Waiting for services..."
    @sleep 30
    @echo "Pulling Ollama model..."
    docker compose -f docker-compose.full-test.yml exec ollama
        ollama pull llama2:7b || true
    @echo "Test infrastructure ready!"

test-infra-down:
    @echo "Stopping test infrastructure..."
    docker compose -f docker-compose.full-test.yml down -v

test-infra-status:
    docker compose -f docker-compose.full-test.yml ps

# ============================================
# COMPREHENSIVE TESTING
# ============================================

.PHONY: test-full test-unit test-integration test-e2e test-
        benchmark test-load test-security

test-full: test-infra-up
    @echo "Running ALL tests (no skips)..."
    set -a && source .env.full-test && set +a && \
    go test -v -count=1 -timeout=30m ./...
    go test -v -count=1 -tags=integration -timeout=30m ./...
    cd tests/e2e/challenges && go run cmd/runner/main.go -all
    @echo "All tests completed!"

test-unit:
    @echo "Running unit tests..."
    go test -v -count=1 -timeout=10m ./internal/...

test-integration: test-infra-up
    @echo "Running integration tests..."
```

```
    set -a && source .env.full-test && set +a && \
    go test -v -count=1 -tags=integration -timeout=20m ./...

test-e2e: test-infra-up
    @echo "Running E2E tests..."
    set -a && source .env.full-test && set +a && \
    cd tests/e2e/challenges && go run cmd/runner/main.go -all

test-benchmark:
    @echo "Running benchmark tests..."
    go test -bench=. -benchmem -run=^$ ./...

test-load: test-infra-up
    @echo "Running load tests..."
    go test -v -count=1 -timeout=30m ./test/load/...

test-security: test-infra-up
    @echo "Running security tests..."
    go test -v -count=1 -timeout=20m ./tests/security/...

# ============================================
# COVERAGE
# ============================================

.PHONY: coverage coverage-html coverage-report

coverage:
    @echo "Generating coverage report..."
    go test -coverprofile=coverage.out -covermode=atomic ./...
    go tool cover -func=coverage.out

coverage-html: coverage
    go tool cover -html=coverage.out -o coverage.html
    @echo "Coverage report: coverage.html"

coverage-report:
    @echo "Checking coverage thresholds..."
    @go test -coverprofile=coverage.out ./... && \
    COVERAGE=$$(go tool cover -func=coverage.out | grep total |
        awk '{print $$3}' | sed 's/%//') && \
    echo "Total coverage: $$COVERAGE%" && \
    if [ $$(echo "$$COVERAGE < 85" | bc -l) -eq 1 ]; then \
        echo "ERROR: Coverage below 85% threshold!"; \
```

```
        exit 1; \
    fi
```

PART 6: SUCCESS CRITERIA

6.1 Test Criteria

- [ ] ZERO skipped tests in any test run
- [ ] 95%+ code coverage for all internal packages
- [ ] 80%+ code coverage for applications
- [ ] All 6 test types passing
- [ ] All 6 challenge definitions passing
- [ ] No flaky tests

6.2 Code Criteria

- [ ] No placeholder implementations
- [ ] No TODO/FIXME items (except documented future enhancements)
- [ ] No mock implementations for production features
- [ ] All API endpoints implemented
- [ ] All applications feature-complete

6.3 Documentation Criteria

- [ ] README.md in every package
- [ ] Complete API documentation
- [ ] All tutorials verified and tested

- [ ] User manual covers all features

6.4 Video Course Criteria

- [ ] All 28-32 videos produced
- [ ] All placeholder URLs replaced
- [ ] Transcripts available
- [ ] Certificate generation working

6.5 Website Criteria

- [ ] All links valid
- [ ] Mobile responsive
- [ ] Courses section complete
- [ ] Documentation up-to-date
- [ ] WCAG 2.1 AA compliant

APPENDIX A: ESTIMATED TIMELINE

Phase	Duration	Start	End
Phase 1: Test Infrastructure	2 weeks	Week 1	Week 2
Phase 2: Fix Skipped Tests	2 weeks	Week 2	Week 4
Phase 3: Implement Missing Code	4 weeks	Week 4	Week 8
Phase 4: Application Completion	4 weeks	Week 8	Week 12
Phase 5: Test Coverage	2 weeks	Week 12	Week 14
Phase 6: Documentation	2 weeks	Week 14	Week 16
Phase 7: User Manuals	1 week	Week 16	Week 17

Phase	Duration	Start	End
Phase 8: Video Production	7 weeks	Week 17	Week 24
Phase 9: Website Updates	2 weeks	Week 24	Week 26

Total Duration: 26 weeks (6.5 months)

APPENDIX B: RESOURCE REQUIREMENTS

Hardware

- Development machine with 16GB+ RAM
- GPU for Ollama testing (recommended)
- Microphone for video recording
- Storage: 100GB+ for test artifacts

Software

- Go 1.24+
- Docker & Docker Compose
- Chrome/Chromium
- OBS Studio or Camtasia (video recording)
- Audio editing software

Services

- GitHub Actions (CI/CD)
- Container registry
- Video hosting platform

Document generated: January 8, 2026 Version: 1.0